

Manual do projeto

Conformidade de Contrato da API REST

Proposta, implementação e plano de execução refinado

Guia de alinhamento e consulta para a equipe

Allan Gabriel Almeida Barros

RA 1352524052

Gabriel Sousa Vasconcelos

RA 13526110176

Francisco Miguel de Lemos Sena Ferreira

RA 1352523939

Matheus Henrique Dias Siqueira

RA 1352623492

Planejamento até 30/11/2026

Janela informada: 30/11 a 05/12/2026 | confirmar com os docentes

Como usar este manual

Um documento de alinhamento para apresentar a proposta e orientar o trabalho diário da equipe.

Instituição: Faculdade UNIFG. **Projeto Final A3:** Conformidade de Contrato da API REST. **Disciplinas:** Garantia da Qualidade de Software e Banco de Dados.

Versão 1.0 | 26/09/2026. Revisão do plano fornecido, com leitura dos dois PDFs e inspeção do código local no commit **719304c023a9**. Este manual descreve o trabalho a implementar; não certifica a execução da API nem apresenta resultados de testes medidos.

Leitura	Páginas	Para quê
Visão comum	3-6	Entender o projeto, o sistema, a arquitetura e as ferramentas.
Plano refinado	7-10	Conhecer as correções do plano e os requisitos propostos.
Implementação e testes	11-18	Consultar contrato, testes, banco, evidências e rotina de execução.
Coordenação e entrega	19-22	Acompanhar marcos, responsabilidades, métricas e apresentação.
Consulta rápida	23-24	Entender os termos e localizar as fontes.

Legenda de confiança

Exigência: consta no enunciado acadêmico. **Observado:** encontrado nos arquivos lidos. **Proposta:** recomendação deste manual, ainda a implementar e aprovar pela equipe. **A confirmar:** depende de execução, decisão ou orientação docente.

Três pendências para a primeira semana: o enunciado limita o grupo a 3 alunos, mas a equipe tem 4; não foi fornecida a rubrica de Banco de Dados; o enunciado pede entrega até 30/11, enquanto a solicitação informa 30/11 a 05/12/2026. Manter 30/11 como prazo interno e confirmar os demais pontos com os professores.

O PDF do plano e o texto colado são materiais de planejamento. O PDF “Projeto- A3.pdf” é a referência acadêmica. Suas instruções foram analisadas como requisitos do trabalho, sem executar ações externas, atribuir tarefas em contas ou alterar a aplicação.

A proposta em uma página

Demonstrar, com evidências reproduzíveis, se uma API entrega o comportamento que documenta.

O problema que vamos investigar

Uma API pode funcionar em uma demonstração e ainda falhar quando recebe dados incompletos, tipos desconhecidos ou referências inexistentes. Se a documentação promete um erro 400 e a resposta real é 500, quem integra com a API perde previsibilidade. Nosso tema é verificar essa conformidade e explicar os riscos encontrados.

O que é o projeto A3

Aplicaremos um processo de qualidade à **Order Management API**, uma aplicação já existente de gestão de pedidos. A equipe desenvolverá requisitos verificáveis, testes, validações de contrato, scripts de banco, relatórios e correções justificadas. O sistema existente é o objeto da avaliação, chamado SUT.



A equipe compara comportamento esperado e observado e documenta cada decisão.

O que o projeto entrega

Frente	Resultado esperado
Qualidade de Software	Requisitos e planos; testes em diferentes níveis; análise estática; defeitos rastreáveis; métricas e comparação antes/depois.
Banco de Dados	Proposta de modelagem, dicionário, migrações, integridade, consultas e testes transacionais, sujeita à rubrica da disciplina.
Equipe	Execução documentada e conhecimento distribuído: todos devem conseguir explicar, executar e modificar um teste.

Pergunta central da apresentação: “Que evidências mostram que a API cumpre o contrato, e quais riscos ainda permanecem?” Uma suíte sem falhas é uma evidência delimitada pelo que foi testado, não uma prova de ausência de defeitos.

O sistema que será avaliado

Um backend de pedidos com clientes, produtos, endereços e pagamentos simulados.

Público-alvo do sistema: aplicações e equipes que precisam cadastrar clientes e produtos e acompanhar pedidos. **Público deste manual:** os quatro integrantes do A3. A API usa HTTP e JSON; uma interface gráfica de loja não é necessária ao escopo.

Recurso	Operações observadas	O que representa
Clientes (6)	GET/POST /clients; GET/PUT/DELETE /clients/{id}; POST /clients/{id}/address	Pessoa física ou jurídica e seu endereço.
Produtos (5)	GET/POST /products; GET/PUT/DELETE /products/{id}	Produto físico ou digital, com preço e atributos próprios.
Endereços (5)	GET/POST /address; GET/PUT/DELETE /address/{id}	Dados de endereço, CEP e UF.
Pedidos (6)	GET/POST /orders; GET/DELETE /orders/{id}; PUT /orders/{id}/status e /address	Cliente, itens, pagamento, entrega e situação do pedido.
Pagamentos (6)	GET/POST /payment; GET/PUT/DELETE /payment/{id}; POST /payment/{id}/process	Modalidades CARD, PIX e BOLETO.

Inventário observado: 28 combinações de método e rota nos controllers; 1 método de teste, contextLoads. Isso não mede cobertura: o percentual inicial só poderá ser informado após instrumentação e execução. [S3]

Exemplo didático do fluxo

Cadastrar um produto de R\$ 50; criar um pedido com 2 unidades, cliente e endereço; obter o pagamento criado junto com o pedido; processar esse pagamento; consultar pedido e banco. O total esperado de R\$ 100 é um critério proposto para verificar o cálculo. Não criar um segundo pagamento automaticamente, pois o pedido já recebe um no fluxo atual.

As classes de pagamento contêm lógica local de simulação. O material examinado não mostra integração com adquirente, banco ou cobrança real. A demonstração deve usar dados fictícios e explicar essa limitação.

Como as partes se conectam

Cada camada tem uma responsabilidade; os testes devem observar essas fronteiras.



Contrato OpenAPI orienta os testes; o handler padroniza as respostas de erro.

Parte	Responsabilidade	Exemplo de verificação
Controller + DTO	Receber a requisição, mapear dados de entrada e saída e acionar validações.	CEP inválido é rejeitado; response tem o type correto.
Service	Aplicar regras de negócio e coordenar operações.	Não alterar endereço de pedido já enviado.
Repository + JPA	Consultar e persistir entidades no PostgreSQL.	CPF duplicado viola a restrição aprovada.
PostgreSQL	Armazenar dados e preservar integridade e transações.	Uma operação malsucedida não deixa pedido parcial.
Handler de erros	Converter exceções em respostas HTTP padronizadas.	HTTP 500 e campo status do corpo são iguais.
OpenAPI + Swagger UI	Descrever a interface e permitir sua exploração.	A resposta real obedece ao esquema versionado.

Duas fronteiras que não podemos confundir

Contrato externo: rotas, campos, tipos, obrigatoriedade, códigos HTTP e conteúdo das respostas. **Integridade interna:** relações, constraints, regras e transações. Um JSON pode estar correto no formato e conter um total errado; por isso contrato e negócio precisam de verificações distintas.

O código utiliza herança SINGLE_TABLE para Client, Product e Payment: cada família compartilha uma tabela com uma coluna discriminadora. O campo type do JSON e o discriminador do banco se relacionam, mas são elementos diferentes. [S3]

Linguagens e ferramentas

Separar o que já existe do que será acrescentado evita a impressão de que a automação está pronta.

Tecnologia	Situação	Uso e decisão
Java 25 + Spring Boot 4.1.0	Observado no pom.xml	Linguagem e framework da API. Validar o conjunto executando build e um teste mínimo.
Maven Wrapper + Git	Observado	Compilar e controlar versões; usar o wrapper do projeto.
JPA / Hibernate + PostgreSQL	Observado / versão a fixar	Mapeamento e persistência. README indica PostgreSQL 18; fixar a imagem e confirmar em execução.
springdoc / Swagger / OpenAPI	2.8.5 observado	Documentação da API. Para Boot 4, investigar linha 3.x e registrar a versão efetivamente validada. [S4]
JUnit Jupiter, Mockito, AssertJ	A consolidar	Testes Java e simulação de dependências. Preferir versões gerenciadas pelo Spring Boot.
REST Assured + validador OpenAPI	A implementar	Chamadas HTTP e validação do contrato. Provar suporte ao dialeto OpenAPI escolhido.
Docker + Testcontainers	A configurar	PostgreSQL real e descartável nos testes. Fixar ciclo de vida e isolamento.
JaCoCo + Surefire/Failsafe	A configurar	Cobertura e relatórios. JaCoCo 0.8.14 adicionou suporte oficial a Java 25. [S5]
Análise estática	A escolher e configurar	Ao menos uma ferramenta é exigida. Começar com PMD; complementar com SpotBugs e/ou Sonar conforme viabilidade.
Flyway + SQL + DBeaver	Proposta para BD	Migrações, consultas, inspeção e evidências do modelo.
GitHub Actions, Issues e Projects	Proposta de trabalho	Automação, defeitos e tarefas. YAML descreve o pipeline; JSON/YAML descrevem contratos e configuração.
Schemathesis	Complemento após o núcleo	Geração de testes a partir do contrato; registrar versão, configuração e reprodução.

Ajuste ao plano: evitar fixar “JUnit 5” por hábito: a documentação atual do Boot referencia JUnit 6. Usar JUnit Jupiter e confirmar a versão resolvida pelo projeto. Python é opcional para métricas; testes HTTP externos também podem gerar cobertura Java se a JVM estiver instrumentada. [S6-S7]

Revisão do plano: prioridades

O plano cobre bem a disciplina, mas precisa de controles que tornem os resultados comparáveis e as entregas executáveis.

Prio.	Problema do plano	Refinamento proposto
P0	Baseline aparece depois de ajustes de ambiente e código.	Guardar o commit original antes de qualquer modificação. Separar original, versão executável e versão melhorada.
P0	Contrato é gerado pela mesma implementação avaliada.	Revisar o esquema contra requisitos aprovados e congelá-lo. Não atualizar o contrato automaticamente para acomodar falhas.
P0	Filtro valida qualquer requisição, inclusive inválida.	Nos negativos, permitir o envio inválido e validar a resposta. A falha esperada deve vir da API.
P0	Defeitos candidatos são misturados a convenções REST.	200 em POST não prova quebra de contrato. Definir requisito antes de mudar para 201; idem DELETE 404 ou 204.
P1	Pipeline só chega na quarta semana.	Criar execução mínima na S1 e primeiro teste HTTP na S2, depois de validar seu requisito. Expandir incrementalmente.
P1	Cobertura inicial é estimada como quase zero.	Medir e guardar denominadores. Separar a suíte original da suíte nova executada na versão de referência.
P1	Toda falha gera defeito; há meta de 12-18 defeitos.	Fazer triagem: produto, teste, contrato ou ambiente. Registrar defeitos reais sem impor quantidade artificial.
P1	Sugere manter defeitos leves abertos por realismo.	Corrigir quando viável. Aceitar pendência apenas por decisão documentada de risco e prazo.

O que manter

O foco em rastreabilidade, testes negativos, risco de pedidos/pagamentos, integração com PostgreSQL e comparação antes/depois é adequado. Manter também a revisão por outra pessoa e a participação de todos na automação e na demonstração.

Inspeções e análise estática também produzem achados válidos. Eles exigem evidência verificável no artefato, não necessariamente um teste HTTP. Hipóteses sobre comportamento em execução devem permanecer identificadas até serem reproduzidas.

Revisão do plano: precisão técnica

Detalhes que afetam escopo, esforço e a confiabilidade da demonstração.

Tema	Ajuste recomendado
Compatibilidade	Confirmar JDK, Boot, springdoc, JUnit, Mockito, JaCoCo, Testcontainers e validador juntos. Não prometer que uma troca isolada de versão resolverá tudo.
Descoberta dos testes	Tags classificam testes, mas não configuram o Maven sozinhas. Usar *Test no Surefire e *IT no Failsafe; ligar integration-test e verify e evitar execução duplicada.
Dados e migrações	Separar dados fictícios de teste das migrações permanentes. Não inserir massa de demonstração em V2 de produção por padrão.
Modelo físico	O plano cita dtype, mas o código declara client_type, product_type e payment_type. Conferir esquema real antes de escrever SQL.
Carga de trabalho	As fases somam 226 h; as cargas individuais indicadas somam 228 h. Reconciliar o quadro. Proposta: 226 h de trabalho e 34 h de reserva, total de 260 h.
Conhecimento concentrado	Gabriel não deve ser o único aprovador. Usar revisão cruzada, pares e rotação de quem demonstra a execução.
Congelamento de casos	Congelar escopo de novas funcionalidades no M2. Continuar criando testes para defeitos, regressões e riscos descobertos.
Metas de desempenho	Definir dataset, aquecimento, rotas, número de medições e concorrência antes de usar o limite de 500 ms.
Foco e ferramentas	Atender à rubrica com um núcleo pequeno e estável. Allure, dashboard, banco de métricas e nuvem entram apenas se houver tempo.

Novo achado de inspeção: cálculo do total

Em `OrderService.buildItems`, o preço recebido por `OrderItem` já é `produto.preço × quantidade`. Em `OrderItem.subTotal`, esse valor é multiplicado pela quantidade novamente. Com preço 50 e quantidade 2, o fluxo sugere subtotal 200. **Hipótese de execução a reproduzir:** o esperado proposto é 100. Criar teste com quantidade maior que 1 e definir se `price` significa preço unitário. [S3]

A ausência de `@ApiResponse`, isoladamente, não prova que erros estão ausentes do OpenAPI: o gerador pode inferir respostas do handler. Inspeccionar o JSON efetivamente gerado antes de registrar a lacuna do contrato.

Escopo e requisitos funcionais

Catálogo inicial proposto: revisar a origem e aprovar cada critério antes de usar seu resultado para julgar a API.

ID	Requisito	Critério verificável proposto
RF-01	Cadastrar cliente PF/PJ	Aceitar os dois tipos; rejeitar nome vazio e ausência do documento exigido pelo tipo.
RF-02	Consultar, editar e excluir cliente	Persistir edição; recurso inexistente em consulta gera 404; política de exclusão explicitada.
RF-03	Cadastrar produto físico/digital	Preço positivo; PHYSICAL exige peso; DIGITAL exige link, conforme regra aprovada.
RF-04	Consultar, editar e excluir produto	Consulta reproduz dados persistidos; referência inexistente gera 404.
RF-05	Gerenciar endereços	CEP de 8 dígitos e UF de 2 caracteres; aplicar a mesma regra na associação ao cliente.
RF-06	Criar pedido completo	Exigir itens, pagamento e endereço; referências válidas; falha implica rollback.
RF-07	Calcular e preservar total	Total é soma de preço unitário histórico × quantidade; duas unidades de 50 totalizam 100.
RF-08	Controlar status do pedido	Aceitar somente valores previstos; validar transições pela tabela aprovada.
RF-09	Alterar endereço de entrega	Bloquear alteração em SHIPPED e DELIVERED; preservar o endereço anterior.
RF-10	Registrar pagamento por tipo	Exigir campos de CARD/PIX/BOLETO; impedir um segundo pagamento para o mesmo pedido.
RF-11	Editar pagamento pendente	Permitir apenas PENDING; troca de tipo mantém o pedido e não deixa pagamento órfão.
RF-12	Processar pagamento simulado	Pedido fica PAID somente após aprovação; definir comportamento de repetição e falha.

A confirmar no refinamento: semântica de price; formato dos documentos; rejeição ou precedência quando clientId e client coexistem; comparação entre valor pago e total; transições; exclusão com vínculos; repetição de processamento. Não inferir essas regras só do comportamento atual.

Fora do escopo principal: interface de loja, autenticação completa, gateway financeiro real, hospedagem pública e carga de produção. Corrigir riscos encontrados e justificar limites, sem transformar o A3 em um novo produto.

Requisitos de qualidade e risco

As metas abaixo são propostas da equipe; os mínimos acadêmicos estão na página 22.

ID / atributo	Como verificar	Meta proposta
RNF-01 / clareza	Testes do corpo de erro, Content-Type e status HTTP.	Erros exercitados seguem o esquema e status do corpo = HTTP.
RNF-02 / contrato	Cobrir operações e validar respostas positivas e negativas.	28 operações inventariadas, cenários aplicáveis mapeados; zero violações não justificadas na suíte final.
RNF-03 / eficiência	Massa versionada com 50 pedidos e dados relacionados; ambiente registrado.	Por rota GET selecionada: 20 aquecimentos e 200 medições sequenciais; p95 até 500 ms.
RNF-04 / proteção	Revisar configuração, respostas, logs e massa de teste.	Sem credencial real versionada; sem dado pessoal real; cartão fictício mascarado nas evidências.
RNF-05 / manutenção	JaCoCo e análise estática em escopo fixado.	85% instruções e 80% branches em services/exceptions; nenhum achado grave confirmado sem tratamento.
RNF-06 / reprodução	Repetir instruções em Windows e Linux/CI.	Mesmo contrato, versão e casos; registrar diferenças de ambiente, sem exigir tempos idênticos.

Prioridade dos testes: probabilidade × impacto

Escala proposta: 1 baixo, 2 médio, 3 alto. Risco 6-9: alto; 3-4: médio; 1-2: baixo. Pontuações são estimativas iniciais e devem ser revistas com os achados.

Área	P × I	Justificativa e ação
Pedidos e pagamentos	3 × 3 = 9	Transações e valores; começar por cálculo, rollback, vínculos e processamento.
Clientes	2 × 3 = 6	Identidade e unicidade; validar duplicidade e comportamento de subtipos.
Produtos	2 × 2 = 4	Preço e type; priorizar limites e campos condicionais.
Endereços	2 × 2 = 4	Endereço incorreto afeta entrega; validar rotas diretas e aninhadas.

Saída proposta: todos os critérios críticos aprovados, mínimo de 95% dos casos executados aprovados e nenhuma falha remanescente sem análise. Informar casos bloqueados/não executados separadamente. Percentual alto não compensa um cálculo financeiro incorreto.

O contrato será nossa referência

O contrato revisado diz o que deveria acontecer; os testes registram o que aconteceu.

Como implementar a verificação

1. Exportar o OpenAPI da versão executável e registrar commit e versão do gerador. 2. Confrontar operações, schemas, required, enum, tipos, formatos, nulidade e respostas com requisitos aprovados. 3. Versionar o contrato revisado. 4. Validar a API contra esse arquivo fixo. 5. Revisar qualquer alteração do contrato em PR com motivo e impacto nos consumidores.

Cenário	Política proposta de contrato
Entrada válida	Validar requisição e resposta, incluindo código HTTP, cabeçalhos relevantes e schema.
Entrada intencionalmente inválida	Não bloquear o envio pelo validador de request. Validar resposta 4xx, corpo de erro e ausência de efeitos no banco.
400 / 404 / 409	400 para entrada/regra inválida; 404 para recurso não encontrado; 409 para conflito de integridade definido.
405 / 415	Tratar método ou mídia não suportados, documentando formato e cabeçalhos aplicáveis.
500 controlado	Simular falha de dependência em ambiente de teste; garantir erro genérico, sem detalhes internos. Sem endpoint de falha em produção.

Consistência do campo type

Definir valores permitidos para clientes, produtos e pagamentos. Modelar variantes com oneOf/discriminator quando suportado pelo validador, ou schemas explicitamente separados. Descrever campos exigidos, ausentes ou nulos por tipo. Testar maiúsculas/minúsculas de acordo com a regra escolhida, além de vazio, null, tipo desconhecido e campos de outro subtipo. [S8]

```
Exemplo de resposta de erro proposta:  
HTTP 400 | Content-Type: application/json  
{ "timestamp": "2026-09-28T10:00:00", "status": 400,  
  "error": "Validation Error",  
  "message": "One or more fields are invalid",  
  "details": ["ZIP code must contain 8 digits."] }
```

No código atual, details é uma lista de mensagens. Adicionar o nome do campo é uma melhoria útil, mas muda o schema: revisar contrato e testes juntos. Não copiar exemplos como evidência de execução.

Estratégia de testes

Cada teste precisa de uma pergunta clara, um resultado esperado e uma evidência recuperável.

Nível / técnica	Implementação proposta	O que prova
Unitário	JUnit Jupiter + Mockito; serviços isolados, sem Spring ou banco.	Decisões e regras locais, incluindo caminhos de falha.
Integração	JPA/JDBC com PostgreSQL em Testcontainers.	Mapeamento, constraints, consultas e transações reais.
Sistema / HTTP	Spring Boot em porta aleatória + REST Assured.	Requisição percorre a aplicação até o banco e volta.
Aceitação	Cenários Dado/Quando/Então associados aos RF.	Critério aprovado está atendido do ponto de vista de uso.
Regressão	Reexecutar casos após correção, preservando vínculo ao defeito.	A correção resolve o caso e não quebra cenários já aprovados.
Caixa-preta	Equivalência, limites e tabela de decisão; casos derivados de requisitos.	Entradas válidas, inválidas e fronteiras relevantes.
Caixa-branca	JaCoCo + grafo de três métodos selecionados.	Decisões e caminhos executados; explicar os não cobertos.

Como evitar testes instáveis

Usar fixtures com dados fictícios próprios, capturar IDs retornados e limpar/recriar o estado de forma determinística. Requisições HTTP em outra thread não são desfeitas automaticamente pela transação do método de teste. Não compartilhar dados mutáveis entre casos; iniciar sem paralelismo até provar isolamento.

Escolher um ciclo de vida para containers e mantê-lo consistente com o contexto Spring. Um container compartilhado por métodos de uma classe não significa compartilhamento seguro entre todas as classes. Testar novamente em base vazia. [S9]

Fuzzing como complemento

Executar Schemathesis contra ambiente descartável. Preparar referências existentes e cenários com estado: uma sequência de 404 não exercita criação/processamento. Registrar versão, contrato, seed quando disponível, orçamento de geração e a requisição mínima que reproduz cada falha. [S11]

Casos, métodos automatizados e invocações parametrizadas são contagens distintas. Um teste de aceitação pode também ser de sistema e regressão; não somar essas categorias como casos diferentes.

Catálogo inicial de cenários

São sugestões para detalhar e aprovar, ainda não executadas. Status inicial de todos: NÃO EXECUTADO.

CT / RF	Cenário	Esperado a validar
001 / 01	Cliente INDIVIDUAL válido	Criar e consultar dados coerentes com o tipo.
002 / 01	Cliente CORPORATE válido	Criar com CNPJ e campos previstos.
003 / 01	Nome vazio	400; sem novo cliente.
004 / 01	Documento obrigatório ausente	400; erro coerente com o tipo.
005 / 01	Documento duplicado	409 conforme política aprovada; sem duplicata.
006 / 02	Cliente inexistente em GET	404 com schema padrão.
007 / 03	PHYSICAL válido	Preço/peso persistidos; type coerente.
008 / 03	DIGITAL válido	Link persistido; campos coerentes.
009 / 03	Preço -0,01 / 0 / 0,01	Rejeitar os dois primeiros; aceitar limite positivo.
010 / 03	Tipo desconhecido ou null	400; sem persistência.
011 / 04	Editar e consultar produto	Alteração persistida e resposta conforme.
012 / 04	Produto inexistente em GET	404 com schema padrão.
013 / 05	CEP de 7 / 8 / 9 dígitos	Só 8 dígitos aceitos com restante válido.
014 / 05	UF de 1 / 2 / 3 caracteres	Somente tamanho 2 aceito pela regra inicial.
015 / 05	CEP com letras	400, mesmo se tiver comprimento 8.
016 / 05	Associar endereço inválido ao cliente	Mesma rejeição da rota de endereço.
017 / 06	Pedido com cliente existente	Criar itens, endereço e pagamento vinculados.
018 / 06	Pedido com cliente novo	Persistir conjunto completo na mesma transação.

Técnicas: CT009, CT013 e CT014 usam valores-limite; CT010 usa particionamento de equivalência; CT004 e CT016 podem compor tabelas de decisão. Para cada variação, registrar os dados exatos e a evidência correspondente.

Sucesso de criação: adotar o status explicitamente aprovado no contrato. O comportamento atual usa 200 em rotas de criação; migrar para 201 e Location é uma decisão de evolução, não um defeito automático.

Catálogo e exemplo de caso completo

Completar os metadados de cada cenário antes de incorporá-lo à execução formal.

CT / RF	Cenário	Esperado a validar
019 / 06	Pedido sem itens	400 e nenhuma gravação parcial.
020 / 06	Produto referenciado não existe	404 e rollback de todo o pedido.
021 / 07	Preço 50; quantidade 2	Total 100; preço unitário preservado.
022 / 07	Alterar preço do catálogo após pedido	Pedido mantém preço histórico.
023 / 08	Status textual desconhecido	400; status anterior preservado.
024 / 08	DELIVERED para PENDING_PAYMENT	Rejeitar conforme tabela de transições aprovada.
025 / 09	Endereço em PROCESSING / SHIPPED	Permitir no primeiro; rejeitar no segundo.
026 / 10	CARD/PIX/BOLETO sem campo exigido	400 por tipo e sem pagamento novo.
027 / 10	Segundo pagamento para um pedido	409; vínculo único preservado.
028 / 11	Editar pagamento PENDING	Persistir alteração válida.
029 / 11	Editar pagamento APPROVED	Rejeitar; pagamento inalterado.
030 / 11	Trocar tipo de pagamento pendente	Vínculo preservado, sem registro órfão.
031 / 12	Processar pagamento válido	APPROVED e pedido PAID.
032 / 12	Repetir processamento	Resultado definido pela política aprovada.
033 / RNF-01	JSON malformatado / ID não numérico	400 padronizado, sem 500 inesperado.
034 / RNF-01	Método não suportado	405 e cabeçalhos aplicáveis.
035 / RNF-01	Falha inesperada controlada	HTTP e corpo 500; mensagem sem dado interno.
036 / RNF-02	Resposta de cada variante de type	Schema e campos específicos consistentes.

Exemplo detalhado: CT013-B - CEP com 7 dígitos

Requisito: RF-05, CA-05.2 (proposto). **Objetivo:** rejeitar limite inferior. **Pré-condição:** API disponível e base isolada. **Dados:** street=Rua Teste; number=10; neighborhood=Centro; city=Recife; state=PE; zipCode=5000000. **Passos:** contar endereços; POST /address com esses dados; inspecionar resposta e recontar. **Esperado:** 400, erro conforme contrato e contagem inalterada. **Obtido:** não medido. **Status:** não executado. **Evidência:** preencher com execução/relatório após rodar.

Banco de Dados como parte do projeto

Proposta para a disciplina de BD; validar entregáveis e profundidade com o professor.



Cliente 1:N Pedidos | Pedido 1:N Itens | Produto 1:N Itens

Pedido - Pagamento: 1:1 no modelo | Endereço associado a cliente e entrega

Modelo conceitual de leitura do código: cliente possui pedidos; pedido possui itens; cada item referencia produto; pagamento referencia pedido; cliente e pedido possuem associações de endereço. As cardinalidades de obrigatoriedade e as constraints físicas devem ser confirmadas no banco criado. [S3]

Entrega	Como desenvolver	Critério de conclusão
DER e dicionário	Separar modelo conceitual e lógico; documentar tipos, PK, FK, nulidade e significado dos campos.	Revisão conjunta com código e esquema real.
Normalização e herança	Analisar dependências, 1FN-3FN, SINGLE_TABLE e preço histórico.	Justificativa do modelo e dos campos nulos por subtipo.
Migrações Flyway	Criar esquema em base vazia; depois usar ddl-auto=validate.	Recriação sem intervenção manual e teste de migração.
Constraints	PK/FK, unicidade, NOT NULL e CHECK condicionais ao tipo.	Inserções inválidas por SQL direto são rejeitadas.
Transações	Provocar falha no meio da criação de pedido e na troca de pagamento.	Não restam gravações parciais ou registros órfãos.
Consultas e índices	6-8 consultas, 2 views propostas e análise de plano.	Resultado conferido com massa conhecida; plano antes/depois documentado.

Consultas sugeridas: faturamento de pagamentos aprovados por período; produtos mais vendidos; pedidos por status; clientes sem pedido; pagamentos pendentes; ticket médio. Definir período, status e unidade monetária para evitar totais ambíguos.

Antes de escrever CHECK, usar os discriminadores reais: `client_type`, `product_type` e `payment_type`. Separar seeds de testes das migrações permanentes. Para dinheiro, propor `BigDecimal` e `NUMERIC` com precisão/escala definidas; alinhar arredondamento, DTO e contrato.

Baseline e ciclo de melhoria

A versão inicial precisa continuar identificável depois das correções.

Referência	O que guardar	Por que existe
B0 - original	Commit original, configuração sem segredos e diagnóstico de execução.	Preserva o ponto de partida antes de qualquer ajuste.
B1 - executável	Correções mínimas para subir, contrato exportado e versões de ferramentas.	Se B0 não executa, permite testar sem apagar a limitação inicial.
B2 - melhorada	Correções funcionais, retestes, contrato revisado e relatórios finais.	Mostra o efeito das melhorias e os riscos residuais.

Duas comparações diferentes

Evolução dos testes: medir a suíte inicial e depois a suíte ampliada sobre a mesma versão da API sempre que possível. **Evolução do produto:** executar a mesma suíte de regressão nas versões de referência e corrigida. Alterações de contrato ou de escopo devem ser identificadas; não comparar percentuais com denominadores diferentes como se fossem equivalentes.

Registro mínimo de uma execução

```
run_id; data/hora; commit_da_api; commit_dos_testes;
versao_do_contrato; hash_do_contrato; JDK; Maven;
imagem_do_PostgreSQL; sistema_operacional;
comando; dataset/seed; testes_executados;
aprovados; falhas; erros; ignorados;
relatorios; defeitos_associados; observacoes
```

Arquivar XML/HTML de testes e cobertura, relatórios de análise estática, requisição/resposta sanitizadas e exportação dos defeitos. Guardar um resumo versionado em reports/ e os relatórios grandes como artefatos ou pacote de entrega. Exportar evidências antes de expirarem no CI.

Ciclo de tratamento de um achado

Registrar a ocorrência, reproduzir, identificar origem (produto, teste, contrato ou ambiente), classificar impacto e urgência, corrigir em branch, revisar, retestar o caso original e executar regressão. Só encerrar com links para correção e evidência. Duplicatas devem apontar ao defeito principal.

Não há meta de “produzir defeitos”. A disciplina exige encontrá-los e demonstrar seu tratamento. Cada achado deve ser real e defensável, inclusive quando originado de revisão documental ou estática.

Organização do repositório e do CI

A estrutura abaixo é proposta; não deve ser confundida com arquivos já implementados.

```
src/main/java/          API existente
src/main/resources/db/migration/ migrações propostas
src/test/java/          unitários, integração e API
src/test/resources/     fixtures e configuração de teste
config/                contratos e regras de análise
docs/                  requisitos, planos e catálogos
reports/               resumos e referências de execução
.github/workflows/ci.yml pipeline proposto
README.md              instalação e execução verificadas
```

Sequência de construção

Passo 1: build reproduzível e um teste unitário descoberto. **Passo 2:** um teste com PostgreSQL descartável. **Passo 3:** uma chamada HTTP válida e uma inválida com validação da resposta. **Passo 4:** cobertura e análise estática. **Passo 5:** ampliar cenários e acrescentar geração de testes por contrato.

Etapa do pipeline	Regra de implementação
Preparar	Checkout, JDK fixado, Maven Wrapper, Docker disponível e configuração de testes.
Verificar	Executar Maven verify; *Test no Surefire; *IT no Failsafe ligado a integration-test e verify. [S10]
Medir	Instrumentar unitários e integração com JaCoCo; consolidar execução sem sobrescrever indevidamente dados.
Analisar	Rodar ferramenta estática com regras versionadas; manter mesma configuração nas comparações.
Publicar evidências	Enviar relatórios mesmo quando houver falhas; preservar falha no status do job.
Validar contrato gerado	Comparar com o contrato aprovado; mudança não autorizada pede revisão, não sobrescrita automática.

Suíte experimental sobre a baseline pode falhar legitimamente e deve guardar esse resultado. Para a branch de entrega, ativar critérios aprovados e bloquear regressões. Não esconder falhas com exclusões amplas ou tolerância global de erro.

Fluxo diário: tarefa com critério claro → branch → implementação/teste → PR → revisão por outro integrante → checks → merge. GitHub Projects organiza trabalho; Issues registram defeitos; o catálogo preserva os casos.

Primeiro dia: guia de execução

Roteiro a validar na S1. Os comandos abaixo não foram executados nesta revisão documental.

1. Preparar e conferir

Instalar Git, JDK 25, uma IDE Java e Docker com containers Linux. Abrir a pasta que contém pom.xml. Confirmar que Maven usa o mesmo JDK da IDE. DBeaver e Postman/Insomnia ajudam a explorar; não são pré-requisitos para rodar a suíte automatizada.

```
java -version
docker version
.\mvnw.cmd -version
.\mvnw.cmd dependency:tree
```

2. Configurar o ambiente local

Na primeira semana, preparar e revisar uma configuração de banco local com versão fixa, banco exclusivo e credencial de desenvolvimento externa ao código. Recomenda-se um arquivo Compose documentado pela equipe. Ele ainda não existe como entrega deste manual. Não reutilizar dados reais ou um banco compartilhado para testes destrutivos.

Após externalizar a configuração, fornecer SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME e SPRING_DATASOURCE_PASSWORD no ambiente local. Conferir host, porta e banco. Dados e logs da aplicação devem apontar ao banco de desenvolvimento; a suíte Testcontainers usa seu próprio banco descartável.

3. Iniciar, explorar e executar

```
Windows: .\mvnw.cmd spring-boot:run
Windows: .\mvnw.cmd verify
Linux/macOS: ./mvnw spring-boot:run
Linux/macOS: ./mvnw verify
```

Com a API disponível, abrir <http://localhost:8080/swagger-ui/index.html> e <http://localhost:8080/v3/api-docs>. verify só representará a suíte completa após configurar plugins e testes. Para selecionar um teste futuro: `.\mvnw.cmd test "-Dtest=NomeDaClasseTest#nomeDoMetodo"`. Para integração: `.\mvnw.cmd verify "-Dit.test=NomeDaClasseIT"`.

Sintoma	Primeira conferência
Java incompatível	JDK usado pelo Maven e versão declarada no pom.xml.
Banco ou container indisponível	Docker ativo; URL/porta e logs; imagem acessível.
Swagger falha ou abre sem schema	Compatibilidade springdoc/Boot e resposta de /v3/api-docs.
Build verde com poucos testes	Contagem esperada, nomes *Test/*IT e execução do Failsafe.

Cronograma refinado

Nove semanas de execução até 29/11; entrega em 30/11 e janela informada até 05/12 sujeita a confirmação.

Período	Entrega e responsável principal	Condição para avançar
S1 28/09-04/10	Gabriel: B0, diagnóstico, matriz de versões e CI mínimo. Francisco/Matheus: consultas acadêmicas.	Situação do grupo, data e BD registradas; build/smoke test ou impedimento reproduzível.
S2 05/10-11/10	Allan: RF/RNF e critérios. Francisco: planos, risco e inspeção. Gabriel: primeiro teste HTTP. Matheus: modelo inicial.	M1: escopo e contrato de referência revisados; um fluxo executável documentado.
S3 12/10-18/10	Todos: catálogo detalhado, primeira análise estática e base de testes. Matheus: migrações iniciais.	Ao menos 30 casos projetados; banco recriado; relatórios iniciais guardados.
S4 19/10-25/10	Gabriel/Allan: expandir testes unitários e HTTP. Matheus: integridade. Francisco: triagem.	Níveis de teste representados e dados isolados; cobertura medida.
S5 26/10-01/11	Todos: negativos, type, transações, aceitação. Matheus: consultas SQL e índices.	Críticos de maior risco exercitados; defeitos confirmados rastreados.
S6 02/11-08/11	Francisco: geração por contrato. Gabriel: caixa-branca. Todos: revisar lacunas.	M2: congelar escopo funcional; preservar suíte de comparação; continuar regressões.
S7 09/11-15/11	Todos: corrigir, retestar e revisar contrato quando necessário.	Ciclo completo de melhoria demonstrável; ausência de pendência crítica sem tratamento.
S8 16/11-22/11	Matheus: métricas. Francisco: consolidação. Todos: revisão cruzada.	M3: relatório completo, comparações justificadas e pacote de evidências.
S9 23/11-29/11	Todos: dois ensaios, execução limpa e revisão final.	Checklist completo; cada integrante demonstra e explica um caso.
30/11 entrega	Consolidar versão final e submeter no canal docente.	Comprovante de envio e pacote acessível; horário a confirmar.

Caminho crítico: ambiente → requisitos e contrato → testes reproduzíveis → correções → retestes → relatório. Se houver atraso, cortar primeiro Allure, dashboard, banco de métricas e nuvem. Preservar os requisitos acadêmicos obrigatórios.

Equipe, papéis e rotina

Distribuição proposta para o kickoff; os nomes e RAs foram fornecidos pela equipe.

Integrante	Frete principal	Revisão e apoio
Allan Gabriel Almeida Barros RA 1352524052	Requisitos, critérios, catálogo, testes de sistema e aceitação.	Revisar cenários e mensagens de erro; apoiar reteste. GitHub a informar.
Gabriel Sousa Vasconcelos RA 13526110176	Ambiente, arquitetura dos testes, CI e caixa-branca.	Parear com todos; compartilhar diagnóstico de ferramentas. GitHub: JkbSousa.
Francisco Miguel de Lemos Sena Ferreira RA 1352523939	Planos, inspeção, análise estática, defeitos e relatório.	Revisar rastreabilidade e implementar testes de contrato. GitHub a informar.
Matheus Henrique Dias Siqueira RA 1352623492	Modelagem, migrações, testes de BD e métricas.	Revisar persistência e reprodução; apoiar CI. GitHub: matheussiqueira-dev.

Capacidade de planejamento

Manter a estimativa original de **226 horas** como ponto de partida, com **34 horas de reserva**: 260 horas de equipe. Referência de capacidade: 65 horas por integrante ao longo de nove semanas, cerca de 7,2 horas por semana. Isso é uma proposta equilibrada de disponibilidade, não uma medição ou compromisso já aceito.

Ritual semanal de 30 minutos

Revisar entregas concluídas com evidência; listar impedimentos; comparar esforço previsto e real; distribuir a próxima semana; atualizar riscos. Limitar cada integrante a uma tarefa principal em execução e combinar apoio para os itens difíceis.

Critério de tarefa concluída

Requisito ou objetivo identificado; alteração revisada por outra pessoa; testes relevantes executados; evidência vinculada; documentação atualizada; pendências explicitadas. Uma tarefa de teste não é concluída apenas por existir um arquivo Java.

Todos devem escrever, revisar e explicar testes. A meta de dez testes por pessoa pode apoiar aprendizagem, mas não deve incentivar duplicatas. Alternar apresentador e revisor, incluindo código escrito por Gabriel.

Métricas que sustentam a conclusão

Nesta versão do manual não há valores medidos. Preencher somente após execução real.

Indicador	Cálculo / origem	Interpretação e limite
Execução do catálogo	Casos executados / casos planejados × 100.	Mostra alcance da execução; informar bloqueados e não executados.
Aprovação	Aprovados / executados × 100; explicitar como falhas e erros são contados.	Não incluir testes ignorados no denominador de executados.
Cobertura de código	Instruções e branches cobertos / totais do escopo. JaCoCo.	Alta cobertura não garante boas asserções ou contrato completo.
Cobertura de contrato	Operações exercitadas / 28; complementar por status e variante de type.	Uma chamada por rota não cobre todos os resultados possíveis.
Defeitos	Únicos confirmados por severidade, corrigidos e abertos. Issues.	Não contar duplicatas, alertas falsos ou falhas de ambiente como bugs do produto.
Densidade	Defeitos de código confirmados / KLOC do código principal.	Fixar ferramenta e exclusões; separar defeitos documentais.
Análise estática	Achados por categoria, complexidade e duplicação.	Usar mesma versão/regras; explicar falso positivo e mudança de escopo.
Desempenho	p95 por rota e taxa de erros, conforme RNF-03.	Registrar máquina, massa e concorrência; amostra local não prova capacidade de produção.

Modelo de comparação a preencher

Medida	Referência	Final	Comentário obrigatório
Cobertura / escopo	Não medido	Não medido	Mesma instrumentação? Mesmos arquivos?
Falhas / casos executados	Não medido	Não medido	Mesma suíte? Houve alteração contratual?
Defeitos críticos abertos	Não medido	Não medido	Quais foram retestados e encerrados?
Complexidade / achados	Não medido	Não medido	Configuração estática foi mantida?

Para cada gráfico ou número: registrar origem, execução e significado. Explicar o que melhorou, o que continua sem evidência e qual decisão a equipe tomou. Não usar números ilustrativos como resultados do projeto.

Entrega, apresentação e ética

O manual orienta a execução; ele não substitui os relatórios finais com evidências reais.

Exigência do enunciado [S1]	Como atender
10 RF e 5 RNF; critérios objetivos	Refinar os 12 RF e 6 RNF propostos; registrar validação de requisitos.
Plano de qualidade e de testes	Cobrir os 11 itens de qualidade; escopo, ambiente, dados e estratégia nos planos.
30 casos e 3 técnicas caixa-preta	Detalhar o catálogo, registrar resultados e demonstrar as técnicas usadas.
Caixa-branca e diferentes níveis	Cobertura, análise de caminhos; unitário, integração, sistema, aceitação e regressão.
Automação e análise estática	Suíte executável e ao menos uma ferramenta estática; CI é recomendado no enunciado.
Defeitos, métricas e melhoria	Registro completo, correções, reteste, regressão e comparação antes/depois.

Pacote final: dez entregáveis

1. Código-fonte e README; 2. plano de qualidade; 3. plano de testes; 4. catálogo; 5. relatório de defeitos; 6. análise estática; 7. relatórios automatizados; 8. métricas; 9. relatório técnico final; 10. apresentação e demonstração. Acrescentar os entregáveis de BD que o professor confirmar.

Roteiro proposto de 18 minutos

0-3 min | Allan: problema, sistema e requisitos. **3-6 min | Gabriel:** arquitetura e execução de um teste. **6-10 min | Allan e Francisco:** entrada inválida, erro e rastreabilidade do defeito. **10-13 min | Matheus:** modelo, constraint e rollback. **13-16 min | Gabriel e Matheus:** cobertura e antes/depois. **16-18 min | Francisco:** riscos, ética e conclusão. Duração sujeita à orientação docente.

Ensaiai também criar um caso, alterar um valor e provocar falha, executar suíte, localizar defeito, explicar uma métrica e justificar risco. Preparar pacote local e instruções para repetir a execução, além das evidências já exportadas.

Responsabilidade e uso de dados

Usar massa sintética; remover segredos e dados pessoais de logs e capturas; preservar autoria e licença da API original; revisar e compreender qualquer código auxiliado por IA. Discutir consequências de falhas: total incorreto, pedido pago indevidamente, perda de integridade e exposição de dados. A ausência de autenticação limita o uso a ambiente controlado; este trabalho não certifica conformidade legal ou operação financeira real.

Glossário para ninguém ficar perdido

Vocabulário comum para as reuniões, os testes e a apresentação.

Termo	Significado no projeto
API REST / endpoint	Interface acessada por HTTP; uma operação combina método e caminho, como GET /orders/{id}.
Request / response / JSON	Requisição enviada, resposta recebida e formato textual usado para transportar os dados.
Contrato / OpenAPI	Descrição formal da interface: operações, campos, schemas e respostas possíveis.
Swagger UI	Página para visualizar e experimentar operações descritas no OpenAPI.
DTO / type	Objeto de entrada ou saída; type indica a variante de cliente, produto ou pagamento.
Bean Validation	Validação declarativa de campos Java; depende de ser acionada no fluxo correto.
SUT	Sistema sob teste: a Order Management API.
RF / RNF / CA	Requisito funcional; requisito de qualidade; critério objetivo para aceitar um requisito.
CT / fixture / mock	Caso de teste; preparação de dados; substituto controlado de uma dependência.
PK / FK / constraint	Chave primária; chave estrangeira; regra de integridade aplicada pelo banco.
Transação / rollback	Unidade de operações; desfazer alterações quando a unidade não pode ser concluída.
Migração / seed	Mudança versionada do esquema; dados conhecidos para preparar um ambiente.
Baseline / regressão	Referência inicial; verificação de que alterações não quebraram comportamentos existentes.
Branch / commit / PR	Linha de trabalho; versão registrada; proposta de alteração revisada pela equipe.
CI / artefato	Execução automatizada de verificações; arquivo produzido, como relatório ou log.
Cobertura / branch	Parte do código exercitada; alternativa de uma decisão do programa.
Severidade / prioridade	Impacto do defeito; urgência para tratá-lo. São classificações distintas.
p95 / KLOC	95º percentil do tempo medido; mil linhas de código sob uma regra de contagem definida.

Se uma tarefa não estiver clara, começar por três perguntas: qual requisito está sendo verificado, que resultado esperamos e qual evidência vai mostrar o resultado?

Fontes e decisões pendentes

Fontes consultadas em 26/09/2026. Documentações online podem mudar; fixar versões na S1.

[S1] Enunciado: Projeto- A3.pdf, 15 páginas. Requisitos nas seções 4-20; stack na 21; rubrica na 23; entrega na 24; reprodução na 25. **[S2] Plano recebido:** A3 - Conformidade de Contrato da API REST.pdf, 23 páginas, e texto colado com o mesmo plano. Foram tratados como propostas a revisar.

[S3] Código local: order-management-api, commit 719304c023a97ec1c585767a9524888f25e09bf6. Inspeccionados README, pom.xml, controllers, DTOs, entidades, serviços e handler. As observações deste manual são estáticas; não houve execução da suíte nem validação do banco em runtime.

Referência	Link
Repositório oficial	https://github.com/matheussiqueira-dev/Conformidade-de-Contrato-da-API-REST
API de origem	https://github.com/JkbSousa/order-management-api
S4 / springdoc e compatibilidade	https://springdoc.org/
S5 / JaCoCo e Java 25	https://www.jacoco.org/jacoco/trunk/doc/changes.html
S6 / testes no Spring Boot	https://docs.spring.io/spring-boot/reference/testing/index.html
S7 / JUnit Jupiter	https://docs.junit.org/current/user-guide/
S8 / OpenAPI 3.1	https://spec.openapis.org/oas/v3.1.0.html
S9 / Testcontainers	https://java.testcontainers.org/test_framework_integration/junit_5/
S10 / Maven Failsafe	https://maven.apache.org/surefire/maven-failsafe-plugin/usage.html
S11 / Schemathesis	https://schemathesis.readthedocs.io/en/stable/

Decisões a registrar no kickoff

Aceite do grupo de quatro e prazo; rubrica de BD; papéis e capacidade; matriz de versões; contrato de referência; semântica de preço e pagamento; política de status e exclusão; formato de erros; metas de saída. Cada decisão precisa de data, responsável, justificativa e impacto nos testes.

Benefícios estudantis: GitHub Education, JetBrains Student Pack, Azure for Students, Oracle Academy, AWS Educate e Google Cloud for Education são apoios opcionais citados pela equipe. Elegibilidade, créditos e licenças não foram validados neste manual. Não depender de resgate ou hospedagem em nuvem para concluir o núcleo do projeto.